70 Questions & Answers

NumPy | Pandas | EDA | Data Cleaning | Duplicates | Matplotlib
Linear, Polynomial & Logistic Regression | KNN | Metrics

Interview + Exam Preparation

Topic	Questions
1. NumPy	Q1 - Q7 (7)
2. Pandas	Q8 - Q15 (8)
3. EDA (Exploratory Data Analysis)	Q16 - Q21 (6)
4. Data Cleaning	Q22 - Q27 (6)
5. Duplicates	Q28 - Q31 (4)
6. Matplotlib	Q32 - Q37 (6)
7. Linear Regression	Q38 - Q44 (7)
8. Polynomial Regression	Q45 - Q49 (5)
9. Logistic Regression	Q50 - Q56 (7)
10. KNN (K-Nearest Neighbors)	Q57 - Q63 (7)
11. Metrics	Q64 - Q70 (7)

1. NumPy

Q1. What is NumPy, and why is it faster than a Python list?

Ans: NumPy is a numerical computing library that provides the **ndarray** (n-dimensional array). It is faster than lists because: (1) data is stored in **contiguous memory**, (2) all elements share the **same dtype**, and (3) operations are **vectorized** in C, so no Python loop is needed.

```
import numpy as np
a = np.arange(1_000_000)
b = a * 2                           # vectorized, very fast
c = [x*2 for x in range(1_000_000)]  # slow Python loop
```

Q2. What are the common ways to create a NumPy array?

Ans: np.array() (from a list), **np.zeros()**, **np.ones()**, **np.arange()** (start, stop, step), **np.linspace()** (evenly spaced points), **np.eye()** (identity matrix), and **np.random.rand/randn/randint()** (random values).

```
np.array([1,2,3]); np.zeros((2,3)); np.arange(0,10,2)
np.linspace(0,1,5); np.eye(3); np.random.randint(0,10,(2,2))
```

Q3. What is broadcasting?

Ans: When an operation involves arrays of different shapes, NumPy automatically stretches the smaller array to match. Rule: compare shapes from the right; each pair of dimensions must be **equal** or one of them must be **1**.

```
a = np.array([[1,2,3],[4,5,6]])   # shape (2,3)
b = np.array([10,20,30])           # shape (3,)
print(a + b)   # b is added to every row
```

Q4. What is the difference between reshape(), flatten() and ravel()?

Ans: reshape() changes the shape (total elements must stay the same; -1 lets NumPy infer a dimension). **flatten()** always returns a **copy** (1D). **ravel()** returns a **view** when possible (faster, saves memory).

```
x = np.arange(6)
x.reshape(2,3); x.reshape(-1,2)
m = x.reshape(2,3); m.flatten(); m.ravel()
```

Q5. What do axis=0 and axis=1 mean?

Ans: axis=0 operates down the rows, giving one result per column. **axis=1** operates across the columns, giving one result per row.

```
m = np.array([[1,2,3],[4,5,6]])
m.sum(axis=0)  # [5 7 9]
m.sum(axis=1)  # [ 6 15]
```

Q6. What are boolean indexing and fancy indexing?

Ans: Boolean indexing filters elements using a condition. **Fancy indexing** selects elements by passing a list or array of indices.

```
a = np.array([5,12,7,20,3])
a[a > 6]         # [12 7 20]
a[[0,2,4]]       # [5 7 3]
a[(a>4)&(a<15)]  # combine conditions with & (not 'and')
```

Q7. What is the difference between a view and a copy? What problem can it cause?

Ans: Slicing returns a **view** (same memory), so changing the view also changes the original array. Use **.copy()** to get an independent copy.

```
a = np.array([1,2,3,4])
v = a[:2]; v[0] = 99        # 'a' changes too!
c = a[:2].copy(); c[0] = 0  # 'a' is safe
```

2. Pandas

Q8. What is the difference between a Series and a DataFrame?

Ans: A **Series** is a 1D labeled array (like a single column). A **DataFrame** is a 2D table of rows and columns, i.e. a collection of Series where each column can have a different dtype.

```
import pandas as pd
s = pd.Series([1,2,3])
df = pd.DataFrame({'name':['A','B'], 'age':[20,25]})
```

Q9. What is the difference between loc and iloc?

Ans: loc is label-based (uses row/column names; the end of a slice is inclusive). **iloc** is integer position-based (the end of a slice is exclusive).

```
df.loc[0:2, 'age']      # rows 0,1,2 (inclusive)
df.iloc[0:2, 1]         # rows 0,1 (exclusive)
```

Q10. Which functions help you quickly understand a dataset?

Ans: head()/tail() (first/last rows), **shape** (rows, columns), **info()** (dtypes and non-null counts), **describe()** (count, mean, std, min, quartiles, max), **columns**, **nunique()** and **isnull().sum()**.

```
df.head(); df.shape; df.info(); df.describe(include='all')
```

Q11. How do you filter rows by condition in a DataFrame?

Ans: Use a boolean mask. For multiple conditions use **&** (and) and **|** (or), with each condition in brackets. **isin()** and **query()** are also handy.

```
df[df['age'] > 22]
df[(df['age']>22) & (df['city']=='Delhi')]
df[df['city'].isin(['Delhi','Pune'])]
df.query('age > 22 and city == "Delhi"')
```

Q12. How does groupby() work?

Ans: It follows the Split-Apply-Combine pattern: split the data into groups, apply an aggregation (sum, mean, count, ...) to each group, then combine the results.

```
df.groupby('city')['salary'].mean()
df.groupby(['city','dept']).agg({'salary':['mean','max'], 'age':'count'})
```

Q13. What is the difference between merge(), concat() and join()?

Ans: merge() is an SQL-style join on common column(s) (how = inner/left/right/outer). **concat()** stacks DataFrames along rows (axis=0) or columns (axis=1). **join()** joins on the index (a shortcut for merge).

```
pd.merge(df1, df2, on='id', how='left')
pd.concat([df1, df2], axis=0, ignore_index=True)
df1.join(df2, how='inner')
```

Q14. When do you use apply(), map() and lambda?

Ans: map() works element-wise on a Series (with a dict or function). **apply()** runs a function on a Series or DataFrame (axis=1 for row-wise). **lambda** is a small anonymous function. Vectorized operations are faster, so prefer them when possible.

```
df['grade'] = df['marks'].apply(lambda x: 'Pass' if x>=40 else 'Fail')
df['gender'] = df['gender'].map({'M':0,'F':1})
df['total'] = df.apply(lambda r: r['a']+r['b'], axis=1)
```

Q15. How do value_counts() and sort_values() work?

Ans: value_counts() returns the frequency of each unique value (normalize=True gives proportions). **sort_values()** sorts by one or more columns (ascending=False for descending).

```
df['city'].value_counts(normalize=True)
df.sort_values('salary', ascending=False).head(5)
```

3. EDA (Exploratory Data Analysis)

Q16. What is EDA and why is it important?

Ans: EDA is the process of understanding data before modeling: its structure, distributions, missing values, outliers, patterns and relationships. It exposes wrong assumptions and data-quality problems, and guides the choice of features and models.

Q17. What is the difference between univariate, bivariate and multivariate analysis?

Ans: Univariate: one variable (histogram, boxplot, countplot). **Bivariate**: relationship between two variables (scatter plot, grouped bar, correlation). **Multivariate**: more than two variables (heatmap, pairplot, scatter with hue).

Q18. How do you detect outliers? (IQR and Z-score)

Ans: IQR method: IQR = Q3 - Q1. Lower bound = Q1 - 1.5*IQR, upper bound = Q3 + 1.5*IQR; points outside are outliers. **Z-score**: z = (x - mean)/std; |z| > 3 is usually treated as an outlier. Boxplots show them visually.

```
Q1, Q3 = df['x'].quantile([0.25, 0.75])
iqr = Q3 - Q1
mask = (df['x'] < Q1-1.5*iqr) | (df['x'] > Q3+1.5*iqr)
outliers = df[mask]
```

Q19. What is correlation? How do you plot a heatmap?

Ans: Correlation measures the strength of the linear relationship between two numeric variables (-1 to +1). +1 is perfect positive, -1 perfect negative, 0 means no linear relationship. **Correlation does not imply causation.**

```
import seaborn as sns, matplotlib.pyplot as plt
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm')
plt.show()
```

Q20. What are skewness and kurtosis? How do you fix skewed data?

Ans: Skewness measures asymmetry: positive (long right tail) or negative (long left tail). **Kurtosis** measures how heavy the tails are. For right-skewed data, apply a **log, square-root or Box-Cox/Yeo-Johnson** transform.

```
df['income'].skew()
df['income_log'] = np.log1p(df['income'])
```

Q21. How do you check the target variable and class imbalance in EDA?

Ans: For classification, look at the target's distribution. If one class is very rare (e.g. 95% vs 5%) the data is imbalanced and accuracy becomes misleading. Remedies: resampling (SMOTE/undersampling), class_weight='balanced', and metrics like F1, Recall and AUC.

```
df['target'].value_counts(normalize=True)
sns.countplot(x='target', data=df)
```

4. Data Cleaning

Q22. How do you detect missing values?

Ans: isnull()/isna() gives a boolean mask; **.sum()** gives the count per column. Use mean() for the percentage. **info()** and a missingno heatmap also help.

```
df.isnull().sum()
(df.isnull().mean()*100).round(2)   # % missing
```

Q23. What are the ways to handle missing values?

Ans: (1) **dropna()**: remove rows/columns (when very few values are missing or the column is useless). (2) **fillna()**: fill with mean/median/mode/constant. (3) **ffill/bfill**: for time series. (4) **SimpleImputer / KNNImputer**: sklearn imputers that fit into pipelines.

```
df.dropna(subset=['age'])
df['age'] = df['age'].fillna(df['age'].median())
from sklearn.impute import SimpleImputer
SimpleImputer(strategy='most_frequent')
```

Q24. When do you use mean, median or mode imputation?

Ans: Mean: numeric, roughly symmetric data without outliers. **Median**: skewed data or data with outliers. **Mode**: categorical columns. Always **fit the imputer on the training data only** and then transform the test data, otherwise you leak information.

Q25. How do you fix data types? (astype, to_datetime, to_numeric)

Ans: Wrong dtypes break calculations and waste memory. **astype()** does basic conversion, **pd.to_numeric(errors='coerce')** turns invalid values into NaN, and **pd.to_datetime()** converts date strings to datetime.

```
df['price'] = pd.to_numeric(df['price'], errors='coerce')
df['date']  = pd.to_datetime(df['date'])
df['id']    = df['id'].astype(str)
```

Q26. What options do you have for treating outliers?

Ans: (1) **Remove** them (if they are data errors), (2) **Cap/Clip/Winsorize** at percentiles, (3) **Transform** (log), (4) use **robust models/scalers** (tree models, RobustScaler). An outlier is not always wrong, so use domain knowledge.

```
low, high = df['x'].quantile([0.01, 0.99])
df['x'] = df['x'].clip(low, high)
```

Q27. How do you convert categorical data to numeric? (Encoding)

Ans: Label/Ordinal Encoding for ordered categories (Low < Medium < High). **One-Hot Encoding** for unordered categories (pd.get_dummies or OneHotEncoder). Use **drop_first=True** to avoid the dummy variable trap in linear/logistic regression.

```
pd.get_dummies(df, columns=['city'], drop_first=True)
from sklearn.preprocessing import OrdinalEncoder, OneHotEncoder
```

5. Duplicates

Q28. How do you find duplicate rows?

Ans: df.duplicated() returns True/False per row (the first occurrence is False, later copies are True). Use **.sum()** for the count.

```
df.duplicated().sum()
df[df.duplicated()]            # only the duplicate rows
df.duplicated(subset=['email']).sum()
```

Q29. How does drop_duplicates() work? What are keep and subset?

Ans: subset chooses which columns define a duplicate (default: all columns). **keep** can be 'first' (default), 'last', or False (drop every copy).

```
df = df.drop_duplicates()
df = df.drop_duplicates(subset=['email'], keep='last')
df = df.drop_duplicates(keep=False)   # remove all copies
```

Q30. Why are duplicates a problem for machine learning?

Ans: (1) Some records get extra weight, producing a **biased model**. (2) If the same row lands in both train and test, you get **data leakage** and inflated accuracy. (3) Statistics such as mean and count become wrong. Remove duplicates **before** splitting the data.

Q31. How do you handle near-duplicates (different case or extra spaces)?

Ans: 'Delhi', 'delhi ' and 'DELHI' look different to a machine. Standardize text first with **str.strip()**, **str.lower()** and by removing stray characters, then check for duplicates. Fuzzy matching (rapidfuzz/fuzzywuzzy) can also help.

```
df['name'] = df['name'].str.strip().str.lower()
df = df.drop_duplicates(subset=['name','phone'])
```

6. Matplotlib

Q32. How do you create a basic plot in Matplotlib?

Ans: Import **matplotlib.pyplot**, call plt.plot() for a line plot, then add a title and labels and finish with show().

```
import matplotlib.pyplot as plt
x = [1,2,3,4]; y = [2,4,1,5]
plt.plot(x, y, color='red', marker='o', linestyle='--')
plt.title('Demo'); plt.xlabel('X'); plt.ylabel('Y')
plt.show()
```

Q33. Which chart should be used when?

Ans: Line: trends/time series. **Bar**: compare categories. **Histogram**: distribution of one numeric variable. **Scatter**: relationship between two numeric variables. **Boxplot**: spread and outliers. **Pie**: parts of a whole (few categories).

```
plt.bar(cats, vals); plt.hist(data, bins=20)
plt.scatter(x, y); plt.boxplot(data); plt.pie(vals, labels=cats)
```

Q34. What is the difference between the pyplot style and the object-oriented (fig, ax) style?

Ans: The pyplot style (plt.plot) is quick but works on an implicit current figure. The **OO style** (fig, ax = plt.subplots()) gives explicit control over figures and axes and is recommended for multiple plots and complex layouts.

```
fig, ax = plt.subplots(figsize=(8,4))
ax.plot(x, y)
ax.set_title('OO style'); ax.set_xlabel('X')
```

Q35. How do you create multiple plots in one figure (subplots)?

Ans: plt.subplots(nrows, ncols) returns a figure and an array of axes. Use **tight_layout()** to prevent overlap.

```
fig, axes = plt.subplots(1, 2, figsize=(10,4))
axes[0].hist(data); axes[0].set_title('Hist')
axes[1].scatter(x, y); axes[1].set_title('Scatter')
plt.tight_layout(); plt.show()
```

Q36. How do you customize and save a plot? (legend, grid, savefig)

Ans: Use **label=** with **plt.legend()** for legends, **plt.grid(True)** for a grid, **figsize** for size, and **plt.savefig()** to save (call it before show()).

```
plt.plot(x, y1, label='Train'); plt.plot(x, y2, label='Test')
plt.legend(); plt.grid(True)
plt.savefig('plot.png', dpi=300, bbox_inches='tight')
```

Q37. What is the difference between a histogram and a bar chart?

Ans: A **histogram** bins continuous numeric data and shows frequencies (bars touch, x-axis is a numeric range). A **bar chart** compares values across discrete categories (bars have gaps between them).

7. Linear Regression

Q38. What is Linear Regression?

Ans: A supervised learning algorithm that predicts a continuous target by fitting a straight line (or hyperplane) between the features and the target. Simple LR: **y = b0 + b1*x**. Multiple LR: **y = b0 + b1*x1 + b2*x2 + ... + e**.

Q39. What is the cost function (MSE) and how is the best line found?

Ans: Cost = **MSE = (1/n) * sum((y - y_pred)^2)**. The best line has the minimum MSE. Two approaches: **OLS / Normal Equation** (closed form, $w = (X^T X)^{-1} X^T y$) and **Gradient Descent** (iteratively updates weights; better for large data).

Q40. What are the main assumptions of Linear Regression?

Ans: (1) **Linearity** between X and y. (2) **Independence** of errors. (3) **Homoscedasticity**: constant variance of errors. (4) **Normality** of residuals. (5) **No multicollinearity** among features.

Q41. Show complete Linear Regression code in sklearn.

Ans: Split the data, fit the model, predict, and evaluate.

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
model = LinearRegression().fit(X_train, y_train)
y_pred = model.predict(X_test)
print(r2_score(y_test, y_pred))
print(mean_squared_error(y_test, y_pred))
```

Q42. What do the coefficient and intercept mean?

Ans: Intercept (b0): the predicted y when all features are 0. **Coefficient (bi)**: the average change in y for a 1-unit increase in feature xi, holding the other features fixed. Its sign shows the direction of the relationship.

```
model.intercept_, model.coef_
```

Q43. What is multicollinearity and how do you detect it with VIF?

Ans: Multicollinearity occurs when independent features are highly correlated with each other, making coefficients unstable. A **VIF** > 5 (or 10) signals a problem. Fixes: drop a correlated feature, use PCA, or use Ridge regression.

```
from statsmodels.stats.outliers_influence import variance_inflation_factor
[variance_inflation_factor(X.values, i) for i in range(X.shape[1])]
```

Q44. What are Ridge and Lasso regression? How do they prevent overfitting?

Ans: Both are **regularization** techniques that add a penalty to the cost. **Ridge (L2)** penalizes squared weights and shrinks them. **Lasso (L1)** penalizes absolute weights and can set some exactly to 0 (feature selection). **alpha** controls the penalty strength.

```
from sklearn.linear_model import Ridge, Lasso
Ridge(alpha=1.0).fit(X_train, y_train)
Lasso(alpha=0.1).fit(X_train, y_train)
```

8. Polynomial Regression

Q45. What is Polynomial Regression and when is it used?

Ans: When the relationship between X and y is curved (non-linear), we add powers of the features (x^2, x^3, ...): **$y = b0 + b1*x + b2*x^2 + ... + bn*x^n$**. Use it when a straight line underfits.

Q46. Show Polynomial Regression code in sklearn.

Ans: Create new features with **PolynomialFeatures**, then fit a **LinearRegression**.

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

poly = PolynomialFeatures(degree=2, include_bias=False)
X_poly = poly.fit_transform(X)
model = LinearRegression().fit(X_poly, y)
y_pred = model.predict(poly.transform(X_new))
```

Q47. How do you choose the right degree? Underfitting vs overfitting?

Ans: Too low a degree causes **underfitting** (high bias). Too high causes **overfitting** (high variance: high train score, low test score). Try several degrees with cross-validation and pick the one with the best CV/test score. This is the bias-variance tradeoff.

```
for d in range(1, 8):
    # build a pipeline for each degree and run cross_val_score
    pass
```

Q48. Why is Polynomial Regression still called a 'linear' model?

Ans: Because the model is **linear in its coefficients (b0, b1, ...)**, even though it is non-linear in x. If you treat the powers as new features, it is just linear regression.

Q49. What are the scaling and extrapolation issues in Polynomial Regression?

Ans: At high degrees feature values become huge, so use a **StandardScaler** and a **Pipeline**. High-degree polynomials also give wild predictions outside the training range (extrapolation). Regularization (Ridge) helps.

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge
model = make_pipeline(PolynomialFeatures(3), StandardScaler(), Ridge(alpha=1))
```

9. Logistic Regression

Q50. What is Logistic Regression?

Ans: Despite the name, it is a **classification** algorithm. It estimates the **probability** that a sample belongs to class 1 (using the sigmoid function), then applies a threshold (default 0.5) to decide the class. It is the most common baseline for binary classification.

Q51. What is the sigmoid function?

Ans: sigmoid(z) = 1 / (1 + e^{-z}), where z = b0 + b1*x1 + ... It maps any real number to the range (0, 1), so the output behaves like a probability. At z = 0 the output is 0.5.

```
def sigmoid(z):
    return 1 / (1 + np.exp(-z))
```

Q52. What is the cost function of Logistic Regression?

Ans: MSE becomes non-convex here, so we use **Log Loss (Binary Cross-Entropy)**: **J = -(1/n) * sum[y*log(p) + (1-y)*log(1-p)]**. It is convex and heavily penalizes confident wrong predictions. It is minimized with gradient descent.

Q53. What is the difference between predict() and predict_proba()? How do you change the threshold?

Ans: predict() returns the final class (0/1) using a 0.5 threshold. **predict_proba()** returns class probabilities. Changing the threshold trades off precision and recall (e.g. lower it to raise recall in disease detection).

```
proba = model.predict_proba(X_test)[:, 1]
y_pred_custom = (proba >= 0.3).astype(int)
```

Q54. Show complete Logistic Regression code in sklearn.

Ans: Split, scale (optional but good), fit, predict, evaluate.

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)
clf = LogisticRegression(max_iter=1000).fit(X_train, y_train)
y_pred = clf.predict(X_test)
print(accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

Q55. How do you interpret odds, log-odds and coefficients?

Ans: Odds = p/(1-p). Logistic Regression models the **log-odds as linear**: log(p/(1-p)) = b0 + b1*x1 + ... A coefficient bi is the change in log-odds for a 1-unit increase in xi. **exp(bi)** is the odds ratio (the factor by which the odds change).

```
np.exp(clf.coef_)   # odds ratios
```

Q56. How is multiclass handled, and what does the regularization parameter C do?

Ans: Multiclass uses **One-vs-Rest (OvR)** or **Softmax (multinomial)**. In sklearn, **C = 1/lambda**: small C means strong regularization (risk of underfitting), large C means weak regularization (risk of overfitting). The default penalty is L2.

```
LogisticRegression(C=0.5, penalty='l2')
```

10. KNN (K-Nearest Neighbors)

Q57. What is KNN? Why is it called a lazy learner?

Ans: KNN finds the **K closest training points** to a new point. For classification it takes a **majority vote**; for regression it takes the **average** of the neighbors. It is a **lazy learner** because it does no learning during training; it just stores the data and does all the work at prediction time.

Q58. Which distance metrics are used in KNN?

Ans: Euclidean: sqrt(sum((a-b)^2)) (default). **Manhattan**: sum(|a-b|). **Minkowski**: generalization (p=1 Manhattan, p=2 Euclidean). **Hamming** for categorical data. **Cosine** for text/high-dimensional data.

```
KNeighborsClassifier(n_neighbors=5, metric='minkowski', p=2)
```

Q59. How do you choose the value of K?

Ans: Small K (e.g. 1) is noisy and **overfits**. Very large K over-smooths and **underfits**. Use **cross-validation** or an error-vs-K (elbow) plot to pick the best K. For binary classification, use an **odd K** to avoid ties.

```
from sklearn.model_selection import cross_val_score
for k in range(1, 21):
    s = cross_val_score(KNeighborsClassifier(k), X, y, cv=5).mean()
    print(k, round(s, 3))
```

Q60. Why is feature scaling essential in KNN?

Ans: KNN is distance-based. A feature with a large range (salary 0-100000) will dominate the distance while a small-range feature (age 0-100) is almost ignored. Apply **StandardScaler or MinMaxScaler** (fit on train, transform test).

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
model = make_pipeline(StandardScaler(), KNeighborsClassifier(5))
```

Q61. Show KNN Classifier code in sklearn.

Ans: Scale the features, fit, and score.

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler

sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test  = sc.transform(X_test)

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
print(knn.score(X_test, y_test))
```

Q62. What are the pros, cons and the curse of dimensionality for KNN?

Ans: Pros: simple, no training, handles non-linear boundaries, works for classification and regression. **Cons**: slow predictions on large data, memory heavy, needs scaling, sensitive to noise and irrelevant features. **Curse of dimensionality**: with many features all points become almost equally far apart, so distance loses meaning; use feature selection or PCA.

Q63. How does KNN Regressor work? What is weights='distance'?

Ans: KNN Regressor returns the **mean** of the K neighbors' target values. With **weights='uniform'** all neighbors count equally; with **weights='distance'** closer neighbors get more weight (1/distance).

```
from sklearn.neighbors import KNeighborsRegressor
KNeighborsRegressor(n_neighbors=5, weights='distance').fit(X_train, y_train)
```

11. Metrics

Q64. What is a Confusion Matrix?

Ans: A table of classification results: **TP** (correct positive), **TN** (correct negative), **FP** (false positive, Type I error) and **FN** (false negative, Type II error). Most other classification metrics are derived from it.

```
from sklearn.metrics import confusion_matrix
confusion_matrix(y_test, y_pred)   # [[TN, FP],[FN, TP]]
```

Q65. Give the formulas for Accuracy, Precision, Recall and F1-score.

Ans: Accuracy = (TP+TN)/Total.
Precision = TP/(TP+FP): of everything predicted positive, how much was right.
Recall (Sensitivity) = TP/(TP+FN): of all actual positives, how many were found.
F1 = 2*P*R/(P+R): harmonic mean of precision and recall.

```
from sklearn.metrics import precision_score, recall_score, f1_score
precision_score(y_test, y_pred); recall_score(y_test, y_pred)
f1_score(y_test, y_pred)
```

Q66. What is the accuracy paradox? When do Precision and Recall matter?

Ans: With imbalanced data (say 99% negative), a model predicting everything as negative scores 99% accuracy yet is useless. Look at F1/Recall/AUC instead. **Precision** matters for spam filters (don't flag good mail). **Recall** matters for cancer/fraud detection (don't miss real cases).

Q67. What are the ROC curve and AUC?

Ans: The **ROC curve** plots **TPR (Recall)** against **FPR** at different thresholds. **AUC** is the area under that curve: 1 is perfect, 0.5 is random guessing. AUC is threshold-independent and useful for comparing models.

```
from sklearn.metrics import roc_auc_score, roc_curve
roc_auc_score(y_test, clf.predict_proba(X_test)[:,1])
```

Q68. Regression metrics: what is the difference between MAE, MSE and RMSE?

Ans: MAE = mean(|y - y_pred|): less affected by outliers, in original units. **MSE** = mean((y - y_pred)^2): penalizes large errors more. **RMSE** = sqrt(MSE): in original units and easy to interpret.

```
from sklearn.metrics import mean_absolute_error, mean_squared_error
mae = mean_absolute_error(y_test, y_pred)
rmse = mean_squared_error(y_test, y_pred) ** 0.5
```

Q69. What are R-squared and Adjusted R-squared?

Ans: R^2 = 1 - (SSres/SStot): the share of the target's variance the model explains. R^2 never decreases when you add features, so **Adjusted R^2** penalizes useless features: 1 - $(1-R^2)(n-1)/(n-k-1)$.

```
from sklearn.metrics import r2_score
r2_score(y_test, y_pred)
```

Q70. What are train-test split and cross-validation? How do you check for overfitting?

Ans: train_test_split divides the data into train and test sets (stratify=y keeps class ratios in classification). **K-Fold Cross-Validation** splits the data into K parts and trains/tests K times for a more reliable score. **Overfitting** shows up as a very high train score with a much lower test score.

```
from sklearn.model_selection import cross_val_score
scores = cross_val_score(model, X, y, cv=5, scoring='accuracy')
print(scores.mean(), scores.std())
```